

1 Dipendenze

Dependency si rappresentano mediante una linea tratteggiata **unidirezionale** che termina con una freccia dall'oggetto dipendente (*client*) a quello da cui dipende (*provider*).

- Si ha una dipendenza tra due classi se la modifica della definizione della classe provider può causare cambiamenti nella classe client;
- Se provider viene modificato e cambia la sua interfaccia, tale modifica deve coinvolgere anche il client → *effetto domino* per sistemi complessi in caso di modifiche potenzialmente onerose.

1.1 Tipologie di dipendenza

Per distinguere tipologia di dipendenza UML mette a disposizione stereotipi, indicati con keywords tra `|||`:

- `|||use|`: client effettua generico uso di qualcosa della destinazione (*dipendenza d'uso generica*);
- `|||call|`: client invoca un'operazione del provider;
- `|||create|`: client crea istanze del provider.

1.2 Implicazioni

Dipendenze sono individuate solitamente durante progettazione di dettaglio, raramente contribuiscono a modello concettuale:

- Durante testing, correttezza del client è vincolata a quella del provider;
- Client non può essere riusato senza provider;
- Modifiche di provider implicano modifiche di client (*ripple effect* - con troppe dipendenze modificare una classe implica modificarne molte altre).

2 Interfacce e Classi Astratte

2.1 Classi ed Operazioni Astratte

Una classe astratta è una classe che non può essere direttamente istanziata poiché ha **una o più operazioni astratte** (dichiarate ma non implementate), in UML si rappresenta scrivendo nome in corsivo (o con etichetta *abstract*).

Inserisci Immagine

2.2 Interfaccia

- Classe priva di implementazione (**tutte le operazioni sono astratte**);
- Descrive porzione di comportamento visibile, comune ad un insieme di oggetti;
- Una o più classi possono fornire implementazione dell'interfaccia;
- In UML si usa notazione grafica simile alle classi, ma con keyword `|||interface|`;
- Classi che implementano un'interfaccia devono fornire un'implementazione per **tutti** i metodi dichiarati nell'interfaccia.

Inserisci Immagine Idea di base è separare specifica di una funzionalità (interfaccia) dall'implementazione della stessa da parte di una o più classi, con due possibili rappresentazioni: *Inserisci Immagine*

2.3 Classi e Interfacce

Classi hanno due tipi di rapporti con interfacce:

- Classe può **fornire** interfaccia se è sostituibile ad essa (praticamente la implementa);
- Classe può **richiedere** interfaccia se ha bisogno di una sua istanza per funzionare (forma di dipendenza).

2.3.1 Esempio

Inserisci Immagine

2.4 Notazione Compatta Lollipop

- **Pallina (lollipop)** rappresenta interfaccia esposta da una classe (realization);
- **Semicerchio (socket)** rappresenta servizio richiesto (dipendenza).

Inserisci Immagine

3 Vincoli

Attraverso associazioni, molteplicità, attributi e generalizzazioni si esprimono i vincoli più importanti delle classi, UML permette la definizione di vincoli nel diagramma mediante i suoi costrutti base o mediante classi **tra parentesi graffe** in diversi linguaggi:

- linguaggio naturale;
- linguaggio di programmazione;
- linguaggi formali specifici (OCL).

Inserisci Immagine

3.1 Object Constraint Language

Linguaggio di specifica progettato per specificare vincoli nei moduli SW, un'espressione OCL specifica un vincolo (predicato logico) sul sistema che deve assumere valore vero. Gli statements OCL sono costruiti in 3 parti:

- **Contesto** - specifica elemento del modello UML a cui si applica (classe, associazione, attributo);
- **Regola** - specifica tipo di vincolo, può essere invariante **inv** (deve essere sempre vero), preconditione **pre** (deve essere vero prima dell'esecuzione di un'operazione da parte della classe) o postcondizione **post**;
- **Espressione** - definisce il vincolo

3.1.1 Sintassi

context <Contesto> <Regola> : <Espressione>